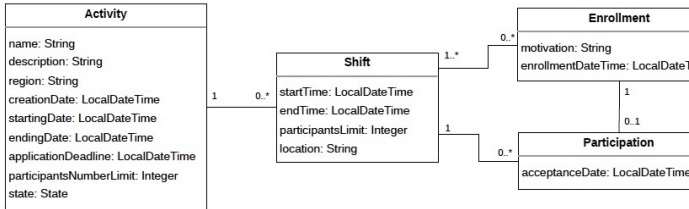


No âmbito da primeira entrega do projeto, todas as tarefas dizem respeito exclusivamente ao código do backend. Devido à refatorização solicitada, o frontend deixará de funcionar. A correção do frontend será tratada apenas na segunda entrega, pelo que não deve ser efetuada qualquer alteração ao frontend nesta fase.

Adicionar Turno à Atividade

Objectivo: Uma atividade está dividida em turnos.

Modelo de Domínio



O turno (Shift) indica o número limite de participantes. Um Enrollment pode ser efetuado para um ou mais turnos da mesma atividade, mas apenas poderá ser selecionado para uma única participação nessa atividade. Participation deixa de estar diretamente associado a Volunteer. Os turnos são criados após a criação da atividade.

De seguida são apresentadas as funcionalidades a desenvolver. **Nota Importante: A introdução do turno vai exigir a adaptação de alguma funcionalidade já implementada. Essa funcionalidade, com os respetivos invariantes, deve ser preservada, a não ser que seja substituída por novas funcionalidades e invariantes.**

Funcionalidades

Serão implementadas as funcionalidades:

- Um membro da instituição adiciona um turno a uma atividade;
- Obter a lista de todos os turnos de uma atividade.

Invariantes

Os seguintes invariantes de domínio devem ser implementados e testados:

1. Os atributos de Shift devem estar definidos, e em particular os do tipo String devem ter pelo menos 20 caracteres e um máximo de 200.
2. A data de início de um Shift deve ser anterior à data de fim do Shift;
3. A data de início e fim de um Shift deve estar compreendida no período definido pela data de início e fim da respetiva atividade;
4. O valor de participantsLimit de um Shift deve ser maior que 0;
5. Apenas podem ser adicionados turnos a atividades no estado APPROVED;
6. O total da soma dos limites de participantes dos Shifts de uma Activity deve ser menor, ou igual, ao limite de participantes da Activity respetiva;
7. Os vários Shifts associados a um Enrollment devem pertencer todos à mesma Activity;
8. Não pode haver dois Shifts associados ao mesmo Enrollment com períodos de execução sobrepostos;
9. O número de participantes num Shift deve ser menor, ou igual, ao seu número limite de participantes;
10. O Enrollment a que o Participation está associado deve ter o Shift do Participation;

Todos os invariantes anteriores devem ser preservados, a não ser que se tenham tomado inconsistentes com os novos invariantes. Pode, no entanto, ser necessário alterar a sua implementação devido à presença da entidade Shift.

As seguintes condições de acesso devem ser implementadas e testadas:

- Apenas um membro da instituição da atividade pode definir um seu turno;
- Qualquer utilizador, registado e não registado, pode obter a lista dos turnos de uma atividade.

Testes

Devem ser realizados os seguintes testes considerando que se pretende obter a máxima cobertura e legibilidade dos testes.

Testes de Domínio

A classe de domínio deve ser testada isoladamente, **usando mocks**. Estes testes devem assegurar que, após a criação, os invariantes de domínio se verificam. O construtor tem a seguinte assinatura:

```
public Shift(Activity activity, ShiftDto shiftDto)
```

Os testes de domínio consideram que Activity contém valores válidos.

Devem ser também alterados os testes de domínio das classes Activity e Enrollment de forma a assegurarem os novos invariantes e preservarem os anteriores (a não ser que se tenham tomado inconsistentes com os novos invariantes).

Testes de Serviço

Devem ser testados os novos serviços:

```
public List<ShiftDto> getShiftsByActivity(Integer activityId)
public ShiftDto createShift(Integer activityId, ShiftDto shiftDto)
```

Nestes testes, deve-se verificar que o serviço devolve os valores corretos. Também deve ser testado que os valores de entrada são válidos: existem uma atividade associada a activityId e um objeto shiftDto.

Estes testes são de componente.

Devem também ser atualizados os testes dos restantes serviços que são afetados por esta alteração.

Testes de Serviço Web

Devem ser testados os serviços web:

```
public List<ShiftDto> getShiftsByActivity(@PathVariable Integer activityId)
public ShiftDto createShift(Principal principal, @PathVariable Integer activityId, @Valid @RequestBody ShiftDto shiftDto)
```

Nestes testes deve ser verificado que o serviço devolve os valores corretos e que os resultados ficam na base de dados. Adicionalmente devem ser efetuados testes para verificar as condições de acesso.

Estes testes são de componente.

Outras APIs

Devido às associações que são definidas e removidas entre as classes Shift, Enrollment e Participation, devem ser implementadas as seguintes APIs:

Página Inicial

Grupos

Avaliação

Bibliografia

Horário

Métodos de Avaliação

Objectivos

Planeamento

Programa

Turnos

Anúncios

Sumários

Pré-requisitos

Componente Laboratorial

Componente de Programação e Computação

Competências Transversais

Princípios Éticos

Notas

Horários de Dúvidas

Aulas de Laboratório

Aulas Teóricas

Código de Conduta para Utilização do Mattermost

Projecto

Primeira Entrega (backend)

```
public Enrollment(Volunteer volunteer, List<Shift> shifts, EnrollmentDto enrollmentDto)
public EnrollmentDto createEnrollment(Integer userId, EnrollmentDto enrollmentDto)
public EnrollmentDto createEnrollment(Principal principal, @Valid @RequestBody EnrollmentDto enrollmentDto)
```

```
public Participation(Enrollment enrollment, Shift shift, ParticipationDto participationDto)
public ParticipationDto createParticipation(Integer shiftId, Integer enrollmentId, ParticipationDto participationDto)
public ParticipationDto createParticipation(@PathVariable Integer shiftId, @PathVariable Integer enrollmentId, @Valid @RequestBody ParticipationDto participationDto)
```

Atenção: estas são as APIs finais, pelo que poderão necessitar de ter outras definições durante tarefas intermédias.

Avaliação

O trabalho encontra-se dividido no seguinte conjunto de tarefas, organizadas em dois grupos: Básicas e Avançadas.

- **Básicas:**
 - Definir a classe Shift, e respetivo construtor, com os seus atributos e associações, sem nenhuma lógica de negócio (invariante) e com os respetivos testes de unidade;
 - Implementar e testar o invariante: A data de início deve ser antes da data de fim à classe e escrever os testes correspondentes;
 - ... uma tarefa para cada um dos outros invariantes de Shift;
 - Definir o método de serviço para criar a classe Shift e respetivos testes de componente;
 - Definir o método de serviço web para criar a classe Shift e respetivos testes de componente;
 - Implementar e testar o serviço de obter os Shifts de uma atividade;
 - Implementar e testar o serviço web de obter os Shifts de uma atividade.
- **Avançadas:**
 - Adicionar a relação entre Shift e Enrollment e garantir que todos os testes continuam a passar (obriga a alterar os respetivos serviços);
 - Implementar e testar o invariante: Não pode haver dois Shifts associados ao mesmo Enrollment com períodos de execução sobrepostos;
 - Remover a relação entre Activity e Enrollment e garantir que todos os testes continuam a passar (obriga a alterar os respetivos serviços);
 - Adicionar a relação entre Shift e Participation e garantir que todos os testes continuam a passar (obriga a alterar os respetivos serviços);
 - Implementar e testar o invariante: O número de participantes num Shift deve ser menor, ou igual, ao seu número limite de participantes;
 - Remover a relação entre Activity e Participation e garantir que todos os testes continuam a passar (obriga a alterar os respetivos serviços);
 - Adicionar a relação entre Enrollment e Participation e garantir que todos os testes continuam a passar (obriga a alterar os respetivos serviços);
 - Implementar e testar o invariante: O Enrollment a que o Participation está associado deve ter o Shift do Participation;
 - Remover a relação entre Participation e Volunteer e garantir que todos os testes continuam a passar (obriga a alterar os respetivos serviços).

A implementação totalmente correta das atividades Básicas garante a aprovação no projeto.

Adicionalmente, devem ser seguidas as seguintes regras:

- Cada tarefa deve preservar o código existente, ou seja, todos os testes, os novos e os regressivos, devem passar;
- Não deve haver código desnecessário na codebase, por exemplo, métodos que nunca são utilizados;
- Não deve haver código redundante na codebase, ou seja, deve-se seguir o princípio do código mínimo: o código mínimo que é necessário para suportar a funcionalidade;
- O código deve respeitar a arquitetura do sistema HumanaEthica e as normas de codificação seguidas, como a utilização de Java Streams, sempre que possível.

Critérios de Avaliação

Para cada tarefa, será avaliada de acordo com os seguintes critérios e respetivas percentagens:

- Tarefa, código, commit e revisão - 25%
- Testes OK - 25%
- Cobertura dos testes - 25%
- Qualidade do código - 25%

Apenas se exige cobertura máxima para o código novo, as alterações ao código existente podem manter o mesmo grau de cobertura.

